

Consumer Use Case: Open-World Decentralized Web Portal

An **open-world decentralized web portal** is a consumer-facing OMI application where a user enters a shared digital space through an ordinary browser, but the objects inside that space are not locked to one server, one app, one account system, or one feed.

The portal behaves like a normal website on the surface:

```
open page → browse objects → inspect details → post/update → share/link → replay history
```

But underneath, every visible object is an addressed OMI state:

```
omi → payload → imo
```

The consumer does not need to understand the full protocol. They experience OMI as a portable portal where pages, cards, files, notes, events, identities, maps, products, posts, and conversations can be inspected in two ways:

```
linear view          = what happened, in what order  
hierarchical view = where it belongs, in what context
```

The open-world portal is therefore not just a webpage. It is a **decentralized object window**.

1. Consumer Story

A user opens the portal on their phone or laptop.

They see a world of objects:

- people
- events
- posts
- notes
- maps
- files
- products
- services
- local groups
- routes
- tasks

- receipts
- claims
- knowledge cards

Each object can be clicked, expanded, followed, copied, shared, verified, or replayed.

To the consumer, the portal feels like a mix of:

```
browser + map + wiki + wallet + canvas + feed + local bulletin board
```

But the key difference is this:

The portal does not merely show content. It shows addressed state.

Every card has an identity. Every edit can produce a receipt. Every object can be viewed as part of a timeline, a graph, a document tree, or a local portal.

2. Open-World Meaning

“Open-world” means the portal assumes the world is not complete.

A normal app usually behaves like a closed database:

```
only this app's users  
only this app's posts  
only this app's IDs  
only this app's permissions  
only this app's feed
```

An OMI open-world portal behaves differently:

```
new objects may appear  
unknown peers may publish  
external files may attach  
local devices may join  
offline state may later sync  
multiple naming systems may coexist  
different portals may project the same object
```

So the portal does not assume it owns the truth. It only asks:

```
Can this object be addressed?  
Can this frame be validated?  
Can this receipt be replayed?  
Can this object be projected safely?  
Can the consumer inspect where it came from?
```

That is the consumer value: the user is not trapped inside one platform's database.

3. The Main Consumer Experience

The portal has four major consumer modes.

3.1 Discover

The user enters through a homepage, map, QR code, shared link, local mesh node, event page, or personal dashboard.

They discover objects as cards:

```
event card  
person card  
place card  
file card  
post card  
route card  
service card  
product card  
discussion card
```

Each card has normal UI fields:

```
title  
summary  
author/source  
time  
location/context  
status  
actions
```

But each card also has OMI fields:

```
omi address
frame receipt
source path
object path
linear replay
hierarchical containment
```

The consumer can ignore those fields until needed.

3.2 Inspect

When the user opens an object, the portal shows two projections.

The **linear projection** answers:

```
What happened?
What came before this?
What does this replay from?
What is the source/object flow?
```

Example:

```
omi → event payload → update payload → comment payload → imo
```

The **hierarchical projection** answers:

```
Where does this belong?
What contains it?
What context controls it?
What unit is being inspected?
```

Example:

```
Community
  Bike Ride
    Route
      Stop
        Comment
```

In OMI notation:

FS → GS → RS → US

The consumer can switch between these views:

View	Consumer Question	Portal Projection
Linear	"What happened?"	replay stream
Hierarchical	"Where is it?"	containment tree
Graph	"What is it connected to?"	object links
Receipt	"Can I trust this?"	validation/replay evidence
Portal	"Can I open it elsewhere?"	portable object projection

3.3 Act

The user can interact with the object:

```
save
share
reply
annotate
verify
join
buy
reserve
donate
route
fork
mirror
publish
```

Each action becomes a new addressed state instead of a hidden platform-side mutation.

For example, joining an event does not merely update a server database. It can produce a small receipt:

```
user X joined event Y at time Z under context C
```

That receipt can be displayed in the user's portal, the event organizer's portal, and any trusted mirror that accepts the same validated object.

3.4 Port

The “portal” part means the object can move across surfaces.

The same object can appear as:

```
web card  
map pin  
JSON Canvas node  
QR code  
SVG badge  
ShadowDOM widget  
local file  
mesh message  
receipt row  
timeline event
```

The object is not identical to its visual rendering. The rendering is only a carrier.

This preserves the OMI rule:

```
carrier ≠ authority  
projection ≠ validation  
glyph ≠ meaning
```

The object’s authority comes from its addressed state, receipts, and validation path.

4. Example Scenario: Local Event Portal

A community bike ride organizer creates an event.

The consumer sees:

```
Sunday Night Lights Ride  
Meetup: Popeyes parking lot  
Route: Vermont Ave Florence → Imperial and back  
Distance: 3.2 miles one way  
Food: hot dogs, hot links, sodas  
Status: active
```

Underneath, the portal treats the event as an OMI object:

```
omi-event
  payload: ride details
  context: local community
  route: map object
  author: organizer identity
  receipts: updates, RSVPs, comments
imo-event
```

A rider opens the event and sees:

Linear view

```
event created
route added
food info added
safety note added
rider joined
speaker note added
time updated
```

Hierarchical view

```
Community
  Sunday Night Lights
    Ride
      Meetup
      Route
      Safety
      Food
      Riders
      Updates
```

Map view

```
start point
turnaround point
route trace
nearby posts
local vendors
live updates
```

Receipt view

```
who posted what  
when it changed  
which frame accepted it  
which portal mirrored it
```

The consumer does not need to know the protocol to benefit. They just get a portable event page that can be opened, mirrored, verified, and replayed.

5. Example Scenario: Personal Knowledge Portal

A user saves a document, a note, and a conversation into their personal OMI portal.

The portal shows the objects as a canvas:

```
Note A → Topic B → File C → Source D
```

The user can inspect the same knowledge in two modes:

Linear replay

```
I found this source  
I wrote this note  
I linked this idea  
I revised this claim  
I shared this object
```

Hierarchical containment

```
Project  
  Topic  
    Claim  
      Evidence  
        Source
```

This is useful because a consumer often needs both:

```
timeline = how I got here
tree     = where this belongs
```

A normal notes app may preserve one or the other. The OMI portal preserves both as projections of the same addressed state.

6. Example Scenario: Local Marketplace Portal

A vendor publishes a mobile cart menu.

The consumer sees:

```
Item: Hot dog
Price: $2
Location: event route
Availability: tonight
Payment: cash/donation
```

The OMI object contains:

```
seller identity
item identity
price state
location state
availability state
receipt history
```

If the vendor changes the price or runs out of stock, the update is a new state, not a silent overwrite.

The consumer can inspect:

```
current menu
previous menu
vendor identity
event relationship
location context
receipt trail
```

That creates a decentralized local commerce portal where small vendors do not need a full platform account structure to publish inspectable objects.

7. Trust Model for Consumers

The consumer should not be forced to trust the portal operator blindly.

The portal should show simple trust labels:

```
Accepted
Unverified
Conflicting
Expired
Mirrored
Local-only
Replayed
Rejected
```

A consumer-facing trust panel could say:

```
This object passed frame validation.
This object has 3 receipts.
This object was mirrored by 2 peers.
This object has 1 conflicting update.
This object is local-only and not yet synced.
```

The consumer sees plain language, while the protocol maintains the deeper validation path.

The rule is:

```
simple labels for humans
receipts for machines
replay for verification
```

8. Identity Without Platform Lock-In

In a normal app, identity often means:

```
login with platform account
profile stored on central server
content owned by platform database
```

In an OMI portal, identity should be object-based:

```
person object  
device object  
author key  
local handle  
portal handle  
receipt history  
relationship graph
```

A consumer can have multiple identities:

```
public identity  
local event identity  
vendor identity  
private notebook identity  
device identity
```

The portal does not need to collapse all of them into one global account.

That is important for an open-world system because the same person may participate differently in different contexts.

9. Consumer Safety and Control

A decentralized portal must not mean “anything can run.”

Consumer safety requires strict projection boundaries:

```
render safely  
sandbox portals  
sanitize innerHTML  
isolate ShadowDOM widgets  
show unknown sources  
block unsafe scripts  
separate carrier from authority
```

Floating wormhole nodes are useful because they let objects appear inside portal surfaces, but they must be treated as projection capsules, not trusted execution by default.

A safe consumer portal should default to:

```
read-only projection first
interaction second
execution last
```

10. Why OMI Matters Here

OMI gives the portal a stable object spine.

The consumer sees:

```
card
page
map
timeline
canvas
widget
receipt
```

The system sees:

```
validated frame
x/y address relation
linear projection
hierarchical projection
receipt slot
carrier surface
```

That means the same state can be shown many ways without changing what it is.

This is the main consumer promise:

One object, many portals, inspectable history, portable context.

11. Minimal Consumer Promise

The open-world decentralized web portal should promise five things:

1. **Portability** — objects are not trapped in one app view.
2. **Inspectability** — users can see history, containment, and source.
3. **Replayability** — important actions can be replayed from receipts.

4. **Context** — every object can show where it belongs.
5. **Safety** — projection does not automatically become authority.

In one sentence:

The portal lets consumers browse, trust, and move through decentralized objects as ordinary web pages, while OMI preserves the addressed state, replay order, containment structure, and validation receipts underneath.

12. Consumer-Facing Interface Sketch

Open World Portal
Search / Scan / Open / Nearby / My Objects
Object Card Title: Sunday Night Lights Ride Status: Accepted Source: Local organizer Context: Community / Bike Ride / Route
Views [Linear Replay] [Hierarchy] [Map] [Graph] [Receipts]
Linear Replay created → route added → safety note → riders joined
Hierarchy Community → Event → Route → Stop → Comment
Actions Save Join Share Mirror Verify Export

13. Portal Architecture in One Flow

```

consumer action
  ↓
portal UI
  
```

```
graph TD; A[ ] --> B[object projection]; B --> C[OMI frame / object address]; C --> D[validation + receipt]; D --> E[linear replay stream]; E --> F[hierarchical containment tree]; F --> G[portable carrier output];
```

↓
object projection
↓
OMI frame / object address
↓
validation + receipt
↓
linear replay stream
↓
hierarchical containment tree
↓
portable carrier output

The consumer-facing app should hide the complexity until the user asks to inspect it. The protocol should remain visible enough that power users can verify, export, mirror, and replay objects without depending on a single platform.